

JomCode Coding Bootcamp

Lists, tuples and mutability

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Select a list or tuple for the data's intended behavior.
- 02 Modify lists safely and distinguish aliasing from copying.
- 03 Use a simple comprehension after tracing an equivalent loop.

Plain meaning → technical view → use

PLAIN MEANING

Keep an ordered group of values together.

TECHNICAL VIEW

Lists are mutable sequences; tuples are fixed sequences; aliasing and copying differ.

WHEN YOU USE IT

Display workshops in order without accidentally changing shared data.

TIP

Change a copy and check whether the original list changed.

Worked example 1: List changes



chapter_07_lists_tuples_and_mutability.py

```
1 titles = ["Python"]
2 titles.append("Web")
3 titles.extend(["SQL", "React"])
4 removed = titles.pop()
5 print(titles)
6 print(removed)
```

PREDICT How does extend differ from append?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
["Python", "Web", "SQL"] (Python repr uses single quotes)  
React
```

WHY THIS HAPPENS

append stores its argument as one item. extend iterates over its argument.

Exercise 1: Independent edit

BUILD

Copy ["Python", "SQL"] and append React only to the copy.

CHECK

Original still has two items. Copy has three.

IF STUCK

Assignment alone creates an alias.

Worked example 2: Tuple unpacking



chapter_07_lists_tuples_and_mutability.py

```
1 dimensions = (1280, 720)
2 width, height = dimensions
3 print(width, height)
4 print(dimensions[:1])
```

PREDICT Why does a one-item tuple include a comma?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

1280 720		
(1280,)		

WHY THIS HAPPENS

The comma defines the tuple. Tuple immutability does not make nested mutable objects immutable.

Exercise 2: Safe filter

BUILD

Produce a new list containing positive counts from [0, 2, -1, 4].

CHECK

=> [2,4]. Empty => []. Source unchanged.

IF STUCK

Filter into a new collection rather than deleting during iteration.

Worked example 3: Alias and copy



chapter_07_lists_tuples_and_mutability.py

```
1 original = ["Python"]
2 alias = original
3 copy = original.copy()
4 alias.append("SQL")
5 print(original)
6 print(copy)
```

PREDICT Which variable names share one list?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
['Python', 'SQL']  
['Python']
```

WHY THIS HAPPENS

original and alias reference the same object. copy has a separate outer list.

Exercise 3: Nested copy explanation

BUILD

Show why copying the outer list does not isolate an inner list. Explain a suitable repair.

CHECK

Changing `copied[0]` also changes `original[0]` after a shallow copy.

IF STUCK

Ask which mutable objects each name actually references.

Checkpoint

PROVE IT

Filter and copy a collection without accidental mutation of the source.